

Performance Comparison of Dijkstra's Algorithm and A*

Shortest-path optimization in hub-and-spoke airline networks

Autenia L. Murray

CSCI 7432 | Instructor: Dr. Yao Xu | May 7, 2026

Click this icon
on each slide
to hear the
narration



Abstract

Evaluating the efficiency gap between blind and heuristic search on a 20-airport directed hub-and-spoke graph.

- Graph nodes: Historical top-hub sample.
- Edge weights: Haversine distance (km).
- Test design: 120 OD pairs, 200 timing runs each.
- Finding: 100% path agreement; A* reduced expansions by 74%.

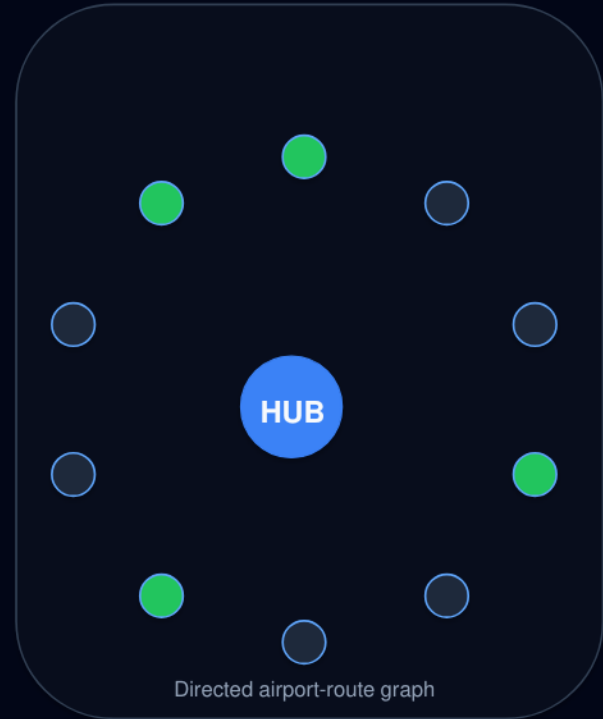
100% accuracy

74% fewer expansions



Introduction

- Airports = Vertices (V); Flights = Directed Edges (E).
- Hub-and-spoke: High traffic concentration at central nodes.
- Goal: Optimize route planning and disruption analysis.
- Research Question: How do Dijkstra and A* compare in accuracy, runtime, and efficiency?



- Hub airport
- Spoke / route node
- High-traffic spoke



Literature Review

Prior research highlights hub-and-spoke robustness and the value of heuristic search.

Siozos-Rousoulis et al. (2021)

Studied U.S. network evolution and hub reliance.

Salem et al. (2022)

Applied Dijkstra to flight scheduling and low-cost selection.

Blanco et al. (2022)

Developed A* methods for flight planning with vertical profiles.

Hart et al. (1968)

Original A* formulation - heuristic guidance without losing optimality.



Algorithm 1: Dijkstra's Algorithm

PSEUDOCODE

```
for each vertex  $v$  in  $V$ :  
     $\text{dist}[v] = \text{infinity}$ ;  $\text{parent}[v] = \text{NIL}$   
 $\text{dist}[s] = 0$ ; insert  $(0, s)$  into Min-PQ  
while PQ is not empty:  
     $u = \text{PQ.extract\_min}()$   
    if  $u == \text{target}$ : stop  
    for each neighbor  $v$  of  $u$ :  
         $\text{alt} = \text{dist}[u] + w(u, v)$   
        if  $\text{alt} < \text{dist}[v]$ :  
             $\text{dist}[v] = \text{alt}$ ;  $\text{parent}[v] = u$   
             $\text{PQ.insert\_or\_update}(\text{alt}, v)$ 
```

"Baseline Single-Source Shortest Path (SSSP)"

- Blind search: no target knowledge.
- Guaranteed optimal for $w \geq 0$.
- Binary heap extraction: $O(\log V)$.

Time: $O((|V| + |E|) \log |V|)$ | Space: $O(|V| + |E|)$



Algorithm 2: A* Algorithm

PSEUDOCODE

```
for each vertex v in V:  
    g[v] = infinity; parent[v] = NIL  
g[s] = 0; insert (h(s), 0, s) into Min-PQ  
while PQ is not empty:  
    u = PQ.extract_min() // using f = g + h  
    if u == target: stop  
    for each neighbor v of u:  
        tentative = g[u] + w(u, v)  
        if tentative < g[v]:  
            g[v] = tentative; parent[v] = u  
            insert (g[v] + h(v), g[v], v) into PQ
```

"Guided Search using $f(v) = g(v) + h(v)$ "

- Guided search: uses $f = g + h$.
- Haversine is admissible ($h \leq h^*$).
- Target-directed search reduces expansions.

$h(v) = \text{Haversine}(v, \text{target})$



Experimentation: Graph & Dataset

20

TOP-HUB NODES

293

DIRECTED EDGES

120

OD TEST PAIRS

200

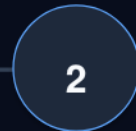
RUNS PER PAIR

- Historical OpenFlights airport and route records.
- Directed adjacency list (flight paths are directional).
- Haversine Great-Circle distance for edge weights.
- Stratified sampling: 40 Short, 40 Med, 40 Long-hauls.

Experimental design



OpenFlights



Graph



Benchmarks



Correctness & Runtime Results

PATH AGREEMENT

100.0% **Correctness Confirmed**

MEAN SPEEDUP

2.44x **Dijkstra Time / A* Time**

A* FASTER (%)

56.7% **A* won overall by mean**

LONG-HAUL WIN

Significant **A* much faster on long routes**

Mean runtime comparison: A* faster on 56.7% of all OD pairs tested.



Node Expansion Results

Average nodes expanded

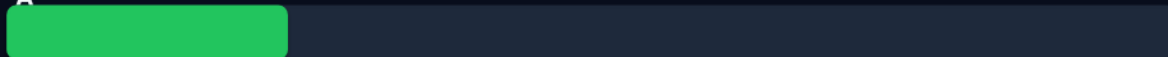
A* reduced mean expansions by about 74%.

Dijkstra



9.325

A*



2.392

- Dijkstra Avg: 9.325 nodes expanded

- A* Avg: 2.392 nodes expanded

- 85.0% of pairs: A* expanded fewer nodes

- 100.0% of Med/Long hauls: A* more efficient



Discussion: Search vs. Overhead

1 Theory vs. Reality

Theoretical efficiency $\neq$ wall-clock runtime.

2 Small Graph Limits

Heuristic math ($\sim 0.01\text{ms}$) is costly at small scale.

3 Search Selective

A* is clearly more efficient in search space.

4 Scaling

Expansion reduction is a strong predictor for large networks.



Conclusion

A* preserves optimality and drastically improves search efficiency, particularly on longer routes.

1

Accuracy: Both algorithms are 100% reliable.

2

Efficiency: A* reduced mean expansions by 74%.

3

Scalability: A* is the superior choice for expanding networks.

4

Next Steps: Rerun benchmark on 50 or 100 airport graphs.



References & Appendices

- Blanco et al. (2022) - ATMOS flight planning studies.
- Cormen et al. (2022) - Introduction to Algorithms.
- OpenFlights (2026) - Airport and airline data source.
- Appendices: Full Python source (astar.py, dijkstra.py) and result CSVs included in package.

Thank you - Questions?

